

BusBuddy Phase 1 Architecture and Accessibility Audit

Domain Isolation, Fare Authority, Service Lifecycle, and Phase 2 Requirements

Independent technical review

19 September 2026

Executive verdict

Phase 1 is directionally strong. Pure-Dart entities, typed failures, one authoritative fare engine, and a shared composition root remove several common sources of divergence. A green 214-test suite is also a credible regression baseline. The architecture is ready to enter Phase 2 only after three contracts are made explicit:

1. **P0 Fare precedence and rounding:** specify whether the VIT Main Gate–Katpadi Railway Station fare is always Rs. 20 before concession or whether hop tiers can override it. Represent money in integer paise and define the rounding point and method.
2. **P0 Singleton ownership and asynchronous teardown:** give every timer, stream subscription, controller, and lifecycle listener one owner; make start/stop idempotent; and make test reset await complete disposal before installing replacements.
3. **P0 Do not cap platform text scaling:** the research document says scaling is clamped to 0.85–2.0. That can reduce or block a user’s accessibility setting. Phase 2 should preserve the platform `TextScaler` and make layouts reflow at large text and display sizes instead.

No Dart source tree was included in the review workspace. Consequently, the items below are risks and acceptance criteria derived from the Phase 1 description and the supplied project documents, not claims that a particular implementation line is defective. A short source inspection should be used to close each item.

1 Evidence and consistency observations

The audit used the supplied implementation research document, updated project handoff, and README. They consistently require deterministic transport facts, a map-independent journey path, meaningful TalkBack output, multimodal feedback, and user control over personalization. Those are the correct architectural anchors.

Two document-level discrepancies should be corrected so that the repository has one trustworthy baseline:

- The README reports 111 passing tests, while the Phase 1 completion statement reports 214. Generate the badge/count from CI rather than maintaining it manually.
- The handoff still describes production implementation and ticketing as pending or future, while Phase 1 reports both as implemented. Mark the handoff as historical or add a dated architecture decision record (ADR) that supersedes it.

2 Question 1: edge cases and runtime risks

2.1 Pure-Dart domain model

Priority	Risk	Required contract or test
P0	Invalid entities can exist	Validate latitude/longitude ranges, non-empty stable IDs, route stop uniqueness/order, non-negative amounts, and ticket time ordering at construction boundaries. Prefer validated factories returning Result over partially valid objects.
P1	Identity is confused with display data	Equality and repository keys should use stable IDs, not stop names, coordinates, route labels, or mutable status. Normalize user-entered names only at search boundaries.
P1	Time and live position ambiguity	Store timestamps in UTC, attach a source timestamp and monotonically increasing sequence/generation, reject out-of-order bus updates, and expose an explicit stale state. Never infer “live” only from stream activity.
P1	Ticket admits illegal transitions	Encode and test a state machine, for example issued → active → used/expired/refunded. Reject activation after expiry, repeated payment callbacks, and transitions from terminal states.
P1	Persisted enum compatibility breaks	Re-exporting enums preserves source compatibility, but persisted enum indexes do not survive reordering. Serialize stable string/wire codes and test unknown future values.
P2	Result is technically handled but failures are lost	Require exhaustive handling at application boundaries. Add flatMap/andThen and error mapping where useful, and translate unexpected exceptions into structured failures at repository boundaries rather than inside entities.
P2	Mutable collections escape entities	Defensively copy route stops and expose unmodifiable views. Otherwise a caller can change a route after fare or journey state has been computed.

For transit routes, explicitly decide how circular routes, repeated physical stops, branches, and reverse travel are represented. A stop ID may legitimately appear twice on a loop; if so, a fare request needs stop *occurrences* or indices, not just two IDs. The domain should also distinguish a physical stop from a route-specific stop occurrence.

2.2 Fare engine and rounding

The current numbers happen to avoid fractional rupees: a 40% concession means paying 60%, producing Rs. 9, Rs. 12, and Rs. 15 for the three stated tiers. That does not remove the need for a money policy; future fees, taxes, or percentage changes will expose binary floating-point and rounding differences.

Recommended authoritative sequence

1. Validate passenger type, route, direction, and ordered stop occurrences.

2. Calculate a non-negative hop count with a documented definition: adjacent stops normally equal one hop.
3. Select exactly one base-fare rule using an explicit precedence table.
4. Apply an eligibility-validated concession using integer arithmetic.
5. Round once, at the final amount, with a named policy; store integer paise.
6. Return a **FareQuote** containing base amount, discount amount, final amount, hop count, applied rule ID/version, and human-readable explanation.

A safe calculation is

$$\text{finalPaise} = \text{roundPolicy} \left(\frac{\text{basePaise} \times \text{payNumerator}}{\text{payDenominator}} \right),$$

using integers/rationals rather than **double**. The product requirements must select the policy (for example, nearest paise with half-up ties); it must not be inherited accidentally from a language primitive.

Mandatory fare decision table and boundary tests

Case	Expected contract
VIT Main Gate ↔ Katpadi Station	Freeze whether this is a special fixed Rs. 20 base or merely an example of the 4–5-hop tier. Test both directions.
0 hops / same stop	Validation failure or explicitly free; never silently classify as 1–3 hops.
1, 3, 4, 5, and 6 hops	Exact boundary tests for Rs. 15, Rs. 20, and Rs. 25.
Unknown or unordered stop	Typed not-found/validation failure; no guessed fare.
Circular/repeated stop	Select occurrences unambiguously and test wrap-around policy.
Student/senior eligibility	Fare engine prices an already validated entitlement, or returns a validation failure; UI labels alone must not grant a concession.
Future fractional result	Golden tests for the chosen paise rounding policy and displayed rupee formatting.
Multiple passengers/refunds	Define whether rounding is per passenger or after aggregation; refund the recorded paid amount, not a recomputation under a newer rule.

Version fare rules. A ticket should retain its quoted rule version and amount so that a later configuration change cannot alter an issued ticket or receipt.

2.3 Service locator lifecycle and races

Replacing approximately 19 local repository/data-source constructions with shared instances removes duplicate simulation loops, but it also concentrates lifecycle risk. Enforce the following ownership model:

- **Composition root owns disposal.** Widgets may subscribe and unsubscribe, but must never dispose locator-owned repositories or app-scoped controllers.
- **Single-flight initialization.** Concurrent getters must await the same initialization future; they must not create two instances during startup, resume, or test override.

- **Idempotent operation.** Repeated `start()`, `stop()`, `dispose()`, `pause`, and `resume` calls must be harmless and covered by tests.
- **Await cancellation.** Stream subscription cancellation may complete asynchronously. Make the reset hook return `Future<void>` and await subscriptions, databases, sockets, and controllers before clearing references or installing overrides.
- **Invalidate late work.** Tag callbacks with a generation/session ID or cancellation token so an old timer, route request, payment callback, or bus stream cannot update newly installed state.
- **Lifecycle policy.** Define which simulation, GPS, audio, socket, and announcement activities pause in background/inactive states and how a fresh snapshot is obtained on resume.
- **No hidden re-creation.** After reset/disposal, a stale widget reference must fail safely; a lazy getter should not silently recreate half of the graph while old controllers still exist.
- **Scoped state.** Keep infrastructure app-scoped, but make journey/session state explicitly replaceable. Prefer constructor injection into screens/controllers over calls to a global locator deep in the widget tree.

Add stress tests for: two simultaneous first reads; rapid start/stop; background/resume; route pop during an in-flight result; override/reset while a timer tick is queued; and repeated test setup/teardown. Run these tests with fake time so failures are deterministic.

2.4 Accessibility-specific race conditions

Shared state can create duplicate or contextless output even when the data is correct. A hidden screen may still listen to a bus stream and announce ETA changes; an old async result may move focus after navigation; and the global assistant overlay may obscure the current control or become interleaved unpredictably in traversal order.

Create one application-level **announcement coordinator**. It should accept semantic events rather than raw data ticks, deduplicate them, prioritize safety/actionable events, suppress output from inactive routes, respect speech/haptic preferences, and never compete with assistant audio. Normal rapidly changing data belongs in a visible text status region; spoken announcements should occur only on meaningful state transitions. Flutter notes that live-region announcements are polite and can be dropped while another accessibility announcement is in progress, so safety-critical alerts also need persistent visual/text state and, where appropriate, haptic or notification fallback.

The floating assistant needs an explicit interaction policy:

- collapsed bubble: one labeled button in a predictable traversal location, never covering the focused control;
- movement: provide non-drag actions such as move left/right/top/bottom or reset position;
- expanded non-modal panel: define how users move between panel and page without focus loss;
- modal mode, if used: trap focus, announce the panel title, restore focus to the invoker on close, and remove obscured content from accessibility traversal;
- full-screen assistant: stop or hide the overlay semantically as well as visually.

3 Question 2: highest-leverage Phase 2 requirements

P0 architecture gates

1. **Publish contracts before widgets.** Add ADRs for fare precedence/rounding, lifecycle ownership, journey state transitions, stale-data handling, announcement policy, and overlay behavior.
2. **Use semantic design tokens.** Define tokens by purpose—for example `color.action.primary`, `color.status.warning`, `space.controlGap`, `size.touchTargetMin`, `text.body`, `motion.reduced`—rather than feature-specific hex values. Every status token needs light, dark, and high-contrast values plus a non-color cue.
3. **Preserve platform accessibility settings.** Do not reduce the system text scale and do not impose a 2.0 ceiling. Use the platform `TextScaler`, responsive constraints, wrapping, scrolling, and content reflow. Treat overflow at large text/display size as a failed test, not a reason to clamp.
4. **Make semantics part of component APIs.** Reusable controls must require an accessible name, role/action, state/value, disabled reason where relevant, and optional hint. Decorative nodes must be excluded; compound cards should avoid duplicate child speech.
5. **Centralize side effects.** Navigation, focus changes, speech, haptics, snackbars, and notifications must be emitted from explicit effects/state transitions, not opportunistically from repeated widget builds.
6. **Adopt explicit asynchronous view states.** Every data surface supports loading, content, empty, stale, offline, permission-denied, recoverable error, and terminal error. Each state exposes the next action in plain language.

P1 accessibility gates

7. **Task-first focus order.** The next journey action comes first; decorative maps and animations do not precede it. On navigation, place focus on a meaningful page heading and restore focus after dialogs. Current Flutter semantics should use `headingLevel` for document structure rather than relying only on the older boolean header flag.
8. **Map equivalence.** Every bus position, route, stop, ETA, direction, selection, and stale/error condition available on the map must also be operable and understandable through a list/card/text path.
9. **48-by-48 is a floor, not the whole policy.** Also enforce spacing, no gesture-only actions, clear disabled states, and alternatives for drag, swipe, long press, and precise map interaction.
10. **Reduced motion and interruption control.** Disable non-essential pulsing, route animation, and auto-scrolling when reduced motion is requested. Never auto-move focus or open a surface because a timer/data update fired.
11. **Plain-language recovery.** Map each structured `Failure` to a stable user message, recovery action, and accessible announcement priority. Do not expose exception strings or silently swallow failure cases.
12. **Privacy and safety boundaries.** Emergency sharing, microphone, location, camera, ticket purchase, and concession proof require explicit purpose, permission timing, confirmation, cancellation, and a non-voice alternative. AI must not initiate payment or present probabilistic recognition as guaranteed identification.

4 Phase 2 verification contract

The 214-test baseline should be extended with a release-blocking accessibility matrix rather than a one-time audit.

Layer	Minimum release evidence
Domain	Fare table/property tests; invalid entity tests; ticket transition tests; UTC/out-of-order/stale bus tests; serialization compatibility tests.
Lifecycle	Fake-time timer tests; double start/dispose; concurrent initialization; awaited override/reset; late callback rejection; background/resume; no listener/timer growth across repeated navigation.
Widget semantics	Automated Android tap-target, labeled-target, and text-contrast guidelines; semantics snapshots for critical controls; no duplicate labels/actions in compound widgets.
Scaling/reflow	Core flow at default, large, and maximum supported platform text/display settings in portrait and landscape; no clipped content, unreachable actions, or two-dimensional scrolling for ordinary forms/cards.
TalkBack	Real Android device traversal for onboarding, route search, booking, active journey, emergency sharing, customization, assistant overlay, permission denial, offline, and stale-data flows.
Multimodal alerts	Verify meaningful-event deduplication, no hidden-screen announcements, speech/TTS contention handling, haptic-off behavior, and persistent text fallback.
User evaluation	Task completion, time, recovery actions, errors, confidence, and qualitative feedback with blind/low-vision participants where ethically feasible; do not substitute blindfolded sighted testing.

Definition of done for every Phase 2 screen

A screen is complete only when it:

- works without color, animation, map imagery, speech, or network as its sole information channel;
- has a deterministic initial focus and logical traversal order;
- remains usable with platform text/display scaling and both portrait and landscape constraints;
- exposes loading, empty, stale, offline, permission, and error recovery states;
- contains no gesture-only operation and meets minimum target size/spacing;
- emits no duplicate, rapid, hidden-route, or contextless announcements;
- passes automated semantics checks and a recorded manual TalkBack checklist.

5 Recommended implementation order

1. Close the three P0 contracts and add their tests without changing visual design.

2. Build the token package and an accessibility-aware component catalog in an isolated gallery, covering buttons, fields, cards, status banners, dialogs, bottom sheets, lists, and map alternatives.
3. Implement the announcement/effect coordinator and route/focus policy before live screens consume shared streams.
4. Rebuild one complete vertical slice—search → route result → active journey—and validate it with TalkBack and maximum text/display settings.
5. Only after the slice passes, migrate ticketing, safety, customization, and the assistant overlay. This prevents inaccessible patterns from being multiplied across the application.

Conclusion

The greatest Phase 2 risk is not visual inconsistency; it is allowing global live state, announcements, overlays, and text-scale overrides to become implicit UI behavior. Keep domain and fare decisions deterministic, make singleton ownership observable and awaitable, and treat semantics/reflow/focus as component contracts. With those gates, the Phase 1 refactor provides a solid foundation for an accessibility-first rebuild.

Sources consulted

- *BusBuddy Implementation Research and UI Design*, supplied file `prism-uploads/BusBuddy_Implementation_Research_and_UI_Design_2.docx`.
- *BusBuddy Project Handoff Document*, supplied file `prism-uploads/BusBuddy_Project_Handoff_Document_Updated_2.docx`.
- Supplied repository overview, `prism-uploads/README.md`.
- Flutter, “Accessibility” and `AccessibilityGuideline` API documentation, accessed 19 September 2026: <https://docs.flutter.dev/ui/accessibility> and https://api.flutter.dev/flutter/flutter_test/AccessibilityGuideline-class.html.
- Flutter API documentation for `TextScaler`, live regions, lifecycle listeners, focus traversal, and semantic heading levels, accessed 19 September 2026: <https://api.flutter.dev/>.
- Dart API documentation for `StreamSubscription.cancel` and `Timer.cancel`, accessed 19 September 2026: <https://api.dart.dev/>.
- W3C, WCAG 2.2 and WCAG2ICT guidance for non-web software, accessed 19 September 2026: <https://www.w3.org/TR/wcag2ict-22/>.